

ECHO — Master Build Playbook

Real-time Dialogue-Continuity Agent | IBM Track (Confluent-powered) | Google Cloud Agentic Cinema Hackathon

This is the single source of truth for the team. Every teammate reads Section 0 and their own section fully before touching code.

0. THE PROJECT IN PLAIN ENGLISH (read this first, no jargon)

The problem

When a movie scene is shot, the actors say the same lines over and over — once in a wide shot, once in a close-up, once more for safety. In the edit room, months later, the editor cuts between these different takes to build the final scene. That only works if the actor said **the exact same words** (or close enough) in every take.

Sometimes actors drift without noticing — Take 1: *"I never trusted him."* Take 3: *"I never really trusted him."* Two extra words. Nobody on set notices, because the AD (Assistant Director) is juggling forty things and there's no time to compare line-by-line. Months later, the editor discovers it doesn't cut cleanly and now they need reshoots, ADR (studio re-recording), or they just live with a worse cut. This costs real money and real time — and it's 100% preventable if someone catches it **the moment it happens, while the actors are still standing there**.

Nobody does this today. It's not because it's not valuable — it's because a human physically cannot listen for this in real time across a 10-12 hour shoot day.

The solution

ECHO listens to the live transcript of every take as it's shot (fed through a live data stream — like a ticker of dialogue lines arriving one at a time), compares each new take against the script and every earlier take of the same scene, and instantly tells the AD: **"This take doesn't match — here's exactly which words changed and why it's a problem."** The AD can then ask for one more take right there on the spot, while it's free, instead of finding out in the edit six weeks later when it's expensive.

Why this isn't "just a chatbot"

We are not asking an AI "hey does this look okay?" We built a pipeline that:

1. **Reads the script** and extracts what each line is *supposed* to be.
2. **Listens to a live stream** of transcribed dialogue, one take at a time, as it arrives — this is real infrastructure, not a text box.
3. **Reasons** about whether a change in wording is harmless (an ad-lib that still cuts fine) or dangerous (breaks continuity).
4. **Shows its work live** on a dashboard — the AD sees it thinking, not just a final answer.
5. **Waits for a human decision** — we never auto-reject a take, a human always has final say.
6. **Produces a report** the editor actually uses later.

That's the difference between a toy and a product.

Who is this for

The **1st Assistant Director (AD)** and **script supervisor** on a film set, during production. Later, the **editor** in post uses the continuity report we generate.

Why we picked the IBM track + Confluent

- IBM's requirement is that we use **IBM Bob** (their dev-time coding agent) while building — like using Copilot/Cursor, but with an approved partner. We use it and keep the session log/commit history as proof.
- **Confluent (Kafka)** is optional for the IBM track, but this is exactly where it's not decorative: dialogue transcripts are inherently a *live, ordered stream of events* (one event = one take's transcript line). We publish each take's transcript as a Kafka message on a topic; our agent is a live **consumer** of that stream, reasoning as messages arrive. If you removed Confluent, you'd have no live pipeline — you'd just have a form where someone pastes text in. That's the whole point: **the partner tech is structurally required, not bolted on.**

The one sentence to say to anyone who asks "what is this"

"ECHO listens to every take as it's filmed and tells the director, in real time, when an actor's line has drifted too far from a previous take to cut together cleanly — catching a problem editors normally don't discover until months later."

1. WHY THIS WINS (judging alignment)

Judging Criterion	How ECHO delivers
Technological Implementation	Real live streaming pipeline (Kafka/Confluent) + Gemini doing genuine semantic reasoning (not string matching) + deployed, working Cloud Run app + visible agent trace in Cloud Logging
Design	Looks like real on-set software (AD dashboard), not a chat window. Live status states, not a spinner.
Potential Impact	Solves a \$ real, named, expensive problem (reshoots/ADR cost real production budget) for a specific named role (AD/script supervisor/editor)
Quality of Idea	Nobody else is targeting <i>dialogue</i> continuity — the entire hackathon's instinct goes to props/wardrobe/visual continuity. We validated this gap structurally (see original strategy doc). It requires real judgment (semantic diff), which is exactly where an LLM belongs instead of a rules engine.

The "Test #8" gut check we already passed: could a judge build this from one generic prompt in a weekend? No — because it requires a live streaming architecture, per-scene state tracking across multiple takes, and a semantic diff classifier tuned specifically for the AD/editor's real decision (cut vs. don't cut), not "is this text similar."

2. THE DETERMINISTIC WORKFLOW (state machine)

This is the backbone. Everything the 4 of you build plugs into this.

```
[1] SCRIPT INTAKE
  → Producer/AD uploads script (PDF/txt) once, before shooting starts
  ↓
[2] REFERENCE EXTRACTION (Gemini)
  → Parses script into scenes → extracts "reference lines" per character per scene
  → Output: structured JSON {scene_id, character, reference_line}
  ↓
[3] LIVE STREAM CONNECT (Confluent/Kafka)
  → Transcript producer publishes one event per take to topic `takes.transcripts`
  → Event = {scene_id, take_number, character, transcribed_line, timestamp, audio_ref}
  ↓
[4] STREAM CONSUME + SEMANTIC DIFF (Gemini, per event, as it arrives)
  → Compare new take's line against:
    (a) the script's reference line
    (b) every PRIOR take of the same scene/character already seen
```

```

→ Classify: MATCH | ACCEPTABLE_ADLIB | DRIFT
→ If DRIFT: extract the exact diverging phrase + a plain-English reason
↓
[5] LIVE STATUS PUBLISH (Firestore)
→ Per-take status written so the dashboard updates instantly:
    Investigating → Comparing to Take N → Classified: DRIFT/MATCH → Awaiting AD
↓
[6] HUMAN APPROVAL GATE (AD/script supervisor, on dashboard)
→ Sees the flagged phrase, prior take it conflicts with, and reason
→ Decides: ACCEPT (log as intentional/acceptable) or FLAG FOR RESHOOT
→ This is a REQUIRED human click – the agent never silently discards a take
↓
[7] SCENE CONTINUITY LEDGER (Firestore + Cloud Storage)
→ Every take's verdict + human decision is logged, timestamped
↓
[8] REPORT GENERATION (fires only after wrap or on-demand)
→ PDF/exportable "Dialogue Continuity Report" per scene for the editor:
    which takes are safe to intercut, which have flagged divergences, why
↓
[COMPLETE]

```

Failure/retry rules (say this explicitly in the demo — judges love seeing you thought about failure):

- If Gemini call times out or Kafka message is malformed → retry once → if still failing, mark NEEDS_REVIEW (never silently pass a take as MATCH by default — fail safe, not fail silent).
- If two takes are ambiguous (could go either way) → Gemini must output a confidence score; below threshold → auto-routes to human with "LOW CONFIDENCE" label instead of guessing.

What's deterministic vs. what's delegated to Gemini:

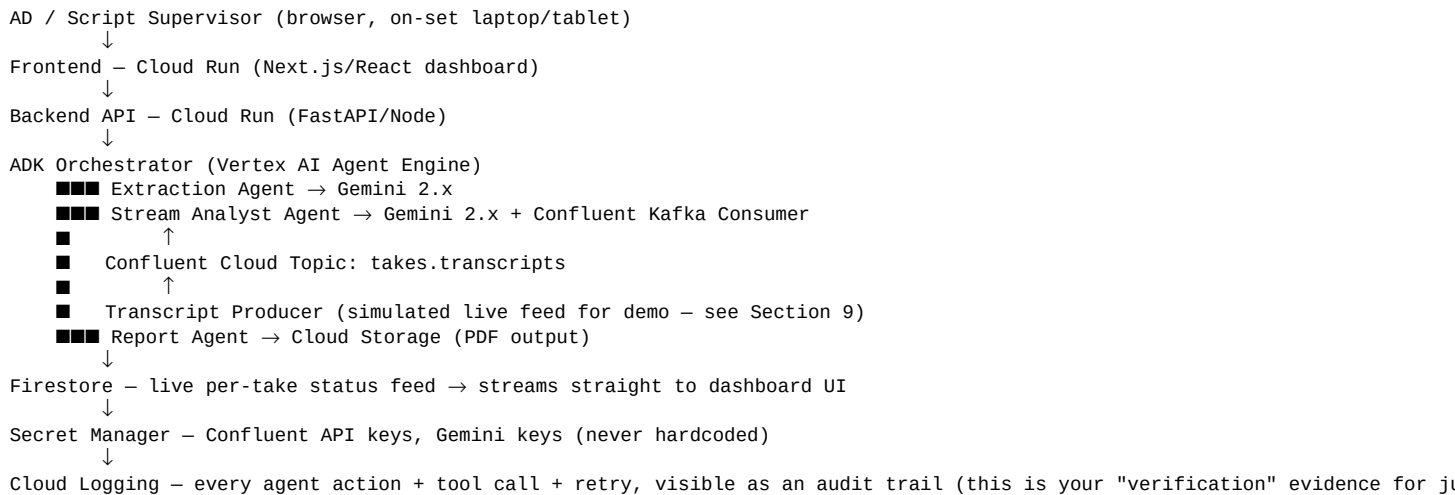
- Deterministic: state transitions, retry logic, approval gate, ledger writes, report generation trigger.
- Gemini's job: (a) script → reference line extraction, (b) semantic classification of MATCH/ACCEPTABLE_ADLIB/DRIFT, (c) plain-English reason generation.

3. AGENT ARCHITECTURE

Agent	Responsibility	Input	Output	Tools
Orchestrator (ADK root)	Owens the state machine end-to-end, sequences everything below	Script upload event; stream events	Final report trigger	ADK, Firestore
Extraction Agent	One-time: turns script into structured per-scene reference lines	Raw script text	JSON reference-line list	Gemini (structured output)
Stream Analyst Agent	The core agent — consumes Kafka topic live, holds per-scene/per-character memory of all prior takes seen, does the semantic diff on each new event	Kafka message (one take transcript)	Classification + phrase-level diff + reason	Gemini, Kafka consumer client, Firestore (state)
Report Agent	Fires after wrap/on approval, compiles the ledger into a human-readable report	Continuity ledger (Firestore)	PDF report	Cloud Storage, PDF generation

No other agents exist. No generic "chat with your script" layer. Every agent has exactly one job on the critical path — this is intentional and you should say so in the demo ("we didn't add agents for the sake of it — each one earns its place").

4. GOOGLE CLOUD + CONFLUENT ARCHITECTURE



Every service has a job — say this in the README, judges specifically look for decoration vs. purpose:

- Cloud Run = real deployed, hosted product (not a notebook).
- Firestore = the thing that makes your dashboard feel "live" instead of "loading."
- Confluent = the only reason this can happen *during* shooting instead of *after*.
- Cloud Logging = your proof the agent actually did the work, not a canned demo.
- Secret Manager = shows you understand real deployment hygiene (small thing, judges notice its absence more than its presence).

5. DATA MODEL

Kafka topic: takes.transcripts

```

{
  "event_id": "evt_00042",
  "scene_id": "SC14",
  "take_number": 3,
  "character": "MARCUS",
  "transcribed_line": "I never really trusted him.",
  "timestamp": "2026-09-09T10:42:11Z",
  "audio_ref": "gs://echo-demo-audio/sc14_take3.wav"
}

```

Firestore: scenes/{scene_id}

```

{
  "scene_id": "SC14",
  "reference_lines": [
    {"character": "MARCUS", "line": "I never trusted him."}
  ],
  "takes": [
    {
      "take_number": 1,
      "character": "MARCUS",
      "line": "I never trusted him.",
      "status": "MATCH",
      "confidence": 0.98
    },
    {
      "take_number": 3,
      "character": "MARCUS",
      "line": "I never really trusted him.",
      "status": "DRIFT",
      "confidence": 0.91,
      "diverging_phrase": "really",
      "conflicts_with_take": 1,
      "reason": "Adds emphasis not present in Take 1 – will not intercut cleanly on this beat."
    }
  ]
}

```

```

    "human_decision": null
  }
]
}

```

Sample script excerpt (use public-domain or self-written — never real copyrighted screenplay)

Write your own 2-3 scene mini-script (5-10 lines of dialogue) with **deliberately planted drift cases** — this is honest synthetic data, not fabricating a fake result. Plant:

- 1 exact match (control case)
- 1 harmless ad-lib ("I never trusted him" → "I didn't trust him") — should classify ACCEPTABLE_ADLIB
- 1 real drift ("I never trusted him" → "I never really trusted him") — should classify DRIFT — **this is your demo's hero case**

6. TEAM STRUCTURE — 4 PEOPLE

Roles are split by **critical-path dependency**, not by "frontend guy / backend guy" — because the whole point of a deterministic workflow is that pieces plug into each other in order. Read your section fully; also skim the others so you understand what's upstream/downstream of you.

■ Person A — Pipeline Lead (Confluent + Stream Analyst Agent) — THE MOST IMPORTANT ROLE

This is the partner-integration linchpin. Nothing else in the demo works without this being solid first — start here, hour zero.

Owns:

- Confluent Cloud account setup, topic creation (takes.transcripts)
- Transcript Producer script (simulates a live on-set transcription feed — for demo purposes this reads from your pre-written sample script + planted drift cases and publishes events with realistic timing/delay, like takes really happening one after another)
- Kafka Consumer wired into the Stream Analyst Agent
- The Stream Analyst Agent itself: Gemini prompt for semantic diff (MATCH / ACCEPTABLE_ADLIB / DRIFT), per-scene memory of prior takes, confidence scoring
- Retry/failure handling for both Kafka and Gemini calls

Deliverables:

1. Working Confluent topic with a Python/Node producer that can publish a take event on command (or on a timer for the live demo)
2. Consumer that reads each event and calls Gemini with a well-tuned prompt (see Section 8)
3. Writes classification result to Firestore in the schema above
4. A local test proving: publish 3 takes → see 3 classifications land correctly (1 MATCH, 1 ADLIB, 1 DRIFT)

Why this is Person A and not split up: if this doesn't work, there is no demo — everything else is UI around this pipeline. This person should have the least other responsibilities so they can go deep.

Stack: Python or Node.js, confluent-kafka client library, Vertex AI Gemini SDK, Firestore SDK, Secret Manager for keys.

■ Person B — Agent Orchestration + Extraction Agent + Cloud Infra

Owns:

- ADK Orchestrator setup (the root agent that sequences everything)
- Extraction Agent: Gemini call that takes the uploaded script and turns it into structured reference lines per scene/character (JSON output)
- Google Cloud project setup: Cloud Run services (frontend + backend), Firestore database structure, Cloud Storage buckets, Secret Manager, Cloud Logging config
- Deployment pipeline (so the whole team can push and see it live, not just on localhost)
- Report Agent: takes the final Firestore ledger and generates the PDF continuity report

Deliverables:

1. Script upload endpoint → Extraction Agent → reference lines saved to Firestore
2. ADK orchestrator that calls Extraction Agent once at intake, then hands off to Person A's Stream Analyst Agent for the live phase
3. Cloud Run deployment live and reachable by a public URL (do this EARLY, day one — don't wait)
4. PDF report generator (can use a simple templating library; content: scene-by-scene table of takes, statuses, reasons, human decisions)
5. Cloud Logging visible and readable — every agent action logged with a clear trace

Why this role matters for judging: this is literally "does it use Google Cloud and ADK properly" — the Technological Implementation score depends heavily on this being real, not a slide.

Stack: Google ADK, Vertex AI Gemini SDK, Cloud Run, Firestore, Cloud Storage, Cloud Logging, a PDF library (e.g., reportlab or pdfkit).

■ Person C — Frontend / AD Dashboard (the thing judges actually watch)

Owns:

- The live dashboard UI: scene list, take-by-take live status column, the "Investigating → Classified → Awaiting AD" visual state machine (see Section 7 for exact UI states — this is not optional, judges specifically dock points for "spinner and final answer only" UIs)
- The side-by-side take comparison view (waveform/transcript for two takes, with the diverging phrase visually highlighted)
- Human Approval Gate UI: Accept / Flag for Reshoot buttons, writes decision back to Firestore
- Live listener on Firestore so the dashboard updates in real time as Person A's pipeline classifies each take (no page refresh — this is the "wow, it's really live" moment)

Deliverables:

1. Dashboard that lists scenes and takes with live-updating status badges
2. The hero comparison view: Take 1 vs Take 3 transcript side by side, diverging words highlighted in red/orange, reason shown in plain English underneath
3. Approve/Flag buttons that write to Firestore and update the ledger

4. Clean, non-chatbot visual design (see Section 7 UX spec) — this single-handedly drives your Design score

Why this role matters for judging: Design is 25% of your score and is the single most-skipped criterion by technical teams who spend all their time on backend. This person's work is what's on screen for 90% of your 3-minute demo — treat it with equal seriousness to the pipeline.

Stack: React/Next.js (Cloud Run hosted), Firestore real-time listeners (onSnapshot), Tailwind or a clean component library, simple audio waveform component (can be a static/simulated waveform if real audio playback is too heavy for time budget — a visual representation is enough, you don't need real audio scrubbing).

■ Person D — Data, Demo Engineering, Devpost/README, IBM Bob compliance

This role is chronically underrated at hackathons and is the difference between a good project and a winning submission. Do not treat this as "the leftover role" — it directly protects your judging score across ALL FOUR criteria.

Owns:

- Writing the sample script + planted drift/ad-lib/match cases (see Section 5) — needs real craft, this IS your demo, get it right
- Building the exact demo timing/script (Section 9) and rehearsing it with the team until it's boring-reliable
- **IBM Bob usage:** use IBM Bob as your actual dev-time coding assistant throughout the build (pairing with whoever needs it that hour), keep session logs/commit co-authorship as proof — this satisfies the hard partner requirement and must not be skipped or faked
- README (Section 12), Devpost writeup (Section 13), LICENSE, repo hygiene (Section 11)
- Risk audit and demo fallback plan (pre-recorded backup clip in case live Confluent call flakes on stage — Section 14)
- QA: running the full pipeline end-to-end repeatedly to catch breakage before submission, filing issues to A/B/C

Deliverables:

1. Final demo script sample data (the exact take transcripts that will be published live on stage)
2. Rehearsed, timed 3-minute demo run-through (do this at least 3 times before submission deadline)
3. Complete README + Devpost page + public repo with LICENSE
4. IBM Bob session evidence in repo (commit log, screenshots, or session export — check exact hackathon requirement on Devpost for what counts as proof)
5. A written fallback plan: "if Confluent call fails live, here's exactly what we do" (cached demo run video, or manual re-trigger)

Why this role matters for judging: Quality of Idea and Potential Impact are largely communicated through README/Devpost narrative and the demo — a brilliant pipeline explained badly scores worse than a good pipeline explained brilliantly. This person is your storyteller and your safety net.

Stack: Markdown, video recording/editing tool for demo backup, IBM Bob (whatever IDE/interface it runs in — check hackathon docs), light scripting for test data generation.

7. UX SPEC — DASHBOARD STATES (Person C, but everyone should read this)

The dashboard must show the agent's live thinking, not a spinner. Use exactly these visible states per take as it processes:

1. "New take received — SC14, Take 3, MARCUS"
2. "Extracting transcript..."
3. "Comparing to script reference line..."
4. "Comparing to Take 1, Take 2..."
5. "Classifying..."
- 6a. "■ MATCH — safe to intercut" (green)
- 6b. "■ ACCEPTABLE AD-LIB — minor wording change, still cuts" (yellow)
- 6c. "■ DRIFT DETECTED — awaiting AD decision" (red, triggers approval gate)

Screens needed:

1. **Scene list view** — all scenes, take counts, overall status (any pending DRIFT flags visible at a glance)
2. **Scene detail / live take feed** — the states above streaming in per take, in order, as they're "shot"
3. **Comparison view** (opens when a DRIFT is flagged) — Take 1 transcript vs Take 3 transcript side by side, diverging words highlighted, reason in plain English, Accept/Flag-for-Reshoot buttons
4. **Continuity ledger / report view** — final scene-by-scene table, exportable to PDF

Do not build a chat box anywhere in this product. There is no "ask the AI a question" input field. Every interaction is either automatic (agent classifying) or a structured decision (Accept/Flag). This is a deliberate design choice — say it explicitly to judges: *"we designed this with zero chat interface on purpose — an AD on set needs status and decisions, not a conversation."*

8. GEMINI PROMPTS (starting point — Person A tunes these)

Extraction Agent prompt (script → reference lines)

You are extracting reference dialogue lines from a film script for a continuity-tracking system.
Input: raw script text for one or more scenes.
For each line of spoken dialogue, output a JSON object:
{ "scene_id": <string>, "character": <string>, "reference_line": <string> }
Return ONLY a JSON array. No other text, no markdown fences.

Stream Analyst Agent prompt (semantic diff, called per incoming take)

You are comparing a newly transcribed film take against the script's reference line and against previously recorded takes of the same line, to determine if this take can be intercut with the others in editing.

Reference line (from script): "{reference_line}"
Prior takes already recorded for this scene/character:
{prior_takes_list}
New take transcript: "{new_take_line}"

Classify the new take as exactly one of:

- MATCH: wording is identical or trivially different (filler words like "uh", "um" only)
- ACCEPTABLE ADLIB: wording differs but meaning and rhythm are close enough to likely cut cleanly against the reference/prior takes
- DRIFT: wording differs in a way that changes emphasis, meaning, or rhythm enough that intercutting with a prior take would look or sound inconsistent

If DRIFT, identify:

- the specific diverging phrase(s)
- which prior take number it most conflicts with
- a one-sentence plain-English reason an Assistant Director would understand

Return ONLY JSON:

```
{
  "status": "MATCH" | "ACCEPTABLE_ADLIB" | "DRIFT",
  "confidence": <0.0-1.0>,
  "diverging_phrase": <string or null>,
  "conflicts_with_take": <int or null>,
  "reason": <string or null>
```

```
}
If confidence is below 0.75, still return your best classification but the system
will route it to human review regardless of status.
```

9. THE 3-MINUTE DEMO — EXACT SCRIPT

Time	What happens	Who drives
0:00–0:10	"In every film shoot, actors repeat the same line across multiple takes so editors can cut between them. When the wording quietly drifts between takes, nobody catches it on set — editors discover it months later, and by then a reshoot is expensive or impossible."	Narrator (Person D)
0:10–0:25	Show the ECHO dashboard — scene list, clean UI, not a chatbot. "This is ECHO — it listens to every take live, on set, and flags dialogue drift before the crew moves on."	Person C drives screen
0:25–0:40	Trigger Take 1 publish to Kafka live on stage. Dashboard shows live states: "New take received → Comparing → Classified: MATCH ■"	Person A triggers producer
0:40–0:55	Trigger Take 2 (acceptable ad-lib). Dashboard shows ACCEPTABLE_ADLIB ■ with reason.	Person A
0:55–1:30	Trigger Take 3 — THE HERO MOMENT. Dashboard flags ■ DRIFT the instant the transcript lands. Comparison view auto-opens: Take 1 "I never trusted him." vs Take 3 "I never really trusted him." — word "really" highlighted, reason shown: "Adds emphasis not present in Take 1 — will not intercut cleanly."	Person C narrates what's on screen
1:30–1:50	AD clicks "Flag for Reshoot" on the dashboard — show the decision write instantly to the ledger.	Person C
1:50–2:10	Cut to Cloud Logging view for 5 seconds — "every one of these decisions is logged and auditable — this is a real agent trace, not a canned response."	Person B
2:10–2:35	Show the final Continuity Report PDF — scene-by-scene, ready for the editor. "This is the artifact that saves the edit six weeks from now."	Person B/D
2:35–2:55	"We built this on the IBM track — IBM Bob paired with us throughout development, and Confluent's live event stream is the only reason this can happen during shooting instead of after. Remove Confluent and this becomes a form you fill out after the fact — too late to matter."	Person D
2:55–3:00	Closing line: "ECHO catches the problem while it's still free to fix."	Person D

Rehearse this exact script at least 3 times end-to-end before the deadline. Time it with a stopwatch. Cut ruthlessly if over.

10. BUILD TIMELINE (~24-28 hours — adjust to your actual clock)

Hour 0–2: Setup (all 4, together)

- Confluent Cloud account + topic created
- Google Cloud project created, billing/APIs enabled (Vertex AI, Cloud Run, Firestore, Secret Manager, Cloud Logging)
- Repo created with structure from Section 11

- IBM Bob environment set up and confirmed working
- Divide and go heads-down

Hour 2–10: Core build (parallel)

- **A:** Producer script working, publishing test events to Kafka; Consumer reading them; first Gemini classification call working end to end (even ugly)
- **B:** Extraction Agent working on sample script; Cloud Run skeleton deployed (even blank page — get the URL live now); Firestore schema created
- **C:** Dashboard skeleton with static/fake data first, wired to real Firestore listener by end of this block
- **D:** Final sample script + drift cases written and validated (does the diff actually produce MATCH/ADLIB/DRIFT correctly when you eyeball it?); README skeleton started

Hour 10–16: Integration

- A + B connect: Extraction Agent's reference lines feed into Stream Analyst Agent's comparison logic
- C connects to real live Firestore data from A's pipeline — first real live demo run, expect it to be broken, that's fine
- D running the pipeline repeatedly, filing bugs immediately to whoever owns that piece
- Approval gate (C) wired to actually write back to Firestore (B/A confirm ledger updates)

Hour 16–20: Report + polish

- B: PDF report generator working off real ledger data
- C: Comparison view polish (highlighting, plain-English reason display), state transition visuals (no bare spinners)
- A: Confidence threshold tuning, retry/failure handling implemented and tested (kill Gemini mid-call, confirm it retries and doesn't crash)
- D: Demo script rehearsal #1, timing it, fixing dead air

Hour 20–24: Hardening + demo prep

- Cut anything not stable — a working 3-agent pipeline beats a broken 5-agent one
- D: Record a backup demo video in case live Confluent call flakes on stage
- D: README + Devpost writeup finalized
- Full team: rehearsal #2 and #3, timed, with the exact final data

Final hours: Submission

- Public repo confirmed accessible, LICENSE present
- Hosted URL confirmed live and working from a fresh browser/incognito
- Devpost submission filled out completely (Section 13)
- IBM Bob usage proof attached/linked

11. REPOSITORY STRUCTURE

```
echo-continuity-agent/
■■■ README.md
■■■ LICENSE
```

(MIT or Apache-2.0 — pick one, add it, don't skip this)

```

■■■■ .env.example
■■■■ frontend/                                (Person C)
■   ■■■■ src/
■   ■   ■■■■ components/
■   ■   ■   ■■■■ SceneList.tsx
■   ■   ■   ■■■■ TakeFeed.tsx
■   ■   ■   ■■■■ ComparisonView.tsx
■   ■   ■   ■■■■ ApprovalGate.tsx
■   ■   ■■■■ lib/firestore.ts
■   ■■■■ package.json
■■■■ backend/                                (Person B)
■   ■■■■ api/                                (Cloud Run FastAPI/Node service)
■   ■■■■ report/                              (PDF generator)
■   ■■■■ requirements.txt
■■■■ agents/                                (Person A + B)
■   ■■■■ orchestrator.py                     (ADK root agent)
■   ■■■■ extraction_agent.py
■   ■■■■ stream_analyst_agent.py
■   ■■■■ report_agent.py
■■■■ streaming/                              (Person A)
■   ■■■■ producer.py                         (simulated on-set transcript feed)
■   ■■■■ consumer.py
■   ■■■■ kafka_config.py
■■■■ data/                                    (Person D)
■   ■■■■ sample_script.txt
■   ■■■■ demo_takes.json
■■■■ infra/                                  (Person B)
■   ■■■■ cloudrun.yaml / deploy.sh
■   ■■■■ firestore_schema.md
■■■■ docs/                                    (Person D)
■   ■■■■ demo_script.md
■   ■■■■ ibm_bob_usage.md                    (proof/log of Bob usage)
■   ■■■■ architecture_diagram.png
■■■■ tests/
    ■■■■ pipeline_e2e_test.py

```

12. README STRUCTURE (Person D fills this in)

```

# ECHO – Real-Time Dialogue Continuity Agent

## The Problem
[2-3 sentences, plain English – use Section 0 of this doc]

## What We Built
[2-3 sentences]

## Why It Matters
[Real cost of dialogue drift – reshoots, ADR cost, editorial time lost]

## Architecture
[Diagram + Section 4 of this doc, condensed]

## Agent Workflow
[Section 2 state machine, condensed]

## Google Cloud Usage
[List every service used and WHY – Section 4 table]

## Partner Usage – Confluent (IBM Track)
[Exact runtime flow – topic name, producer/consumer, why it's structurally required]

## IBM Bob Usage
[Proof: session logs / commit history / screenshots]

## Setup & Run Locally
[Step by step, tested by someone who didn't write the code]

## Demo
[Link to hosted app + link to demo video]

## Limitations & Future Work
[Be honest – judges respect this. E.g., "currently simulates the live transcript feed;
production version would integrate with an on-set transcription vendor's real-time API"]

## License

```

13. DEVPOST STRATEGY

- **Title:** ECHO — Catch Dialogue Drift While It's Still Free to Fix
- **Tagline:** A real-time agent that flags when an actor's line drifts too far between takes to cut together — before the crew moves on.
- **Summary:** 2-3 sentences from Section 0.
- **Problem/Solution/Features:** Pull directly from Sections 0, 2, 7.
- **Architecture:** Pull from Section 4, include the diagram.
- **Google Cloud usage:** Explicit list, one line each, from Section 4.
- **Partner (Confluent/IBM) usage:** Explicit, from Section 4 + 8 — this is where teams lose points by being vague, be exact about topic names and the runtime flow.
- **Impact:** Quantify if you can — e.g., "a single dialogue-drift reshoot can cost a production \$10k-\$50k+ in lost setup/crew time; ECHO catches it in the same take slot, at zero marginal cost."
- **Challenges:** Be honest — e.g., tuning the confidence threshold between ACCEPTABLE_ADLIB and DRIFT was hard and required real judgment calls.
- **Learnings:** What the team actually learned about streaming architectures / semantic diffing / agent design.
- **What's next:** Real transcription vendor integration, multi-language support, integration into existing script supervisor software (e.g., continuity apps already used on set).

14. RISK AUDIT

Risk	Prob.	Impact	Mitigation
Live Confluent call flakes during on-stage demo	Medium	High	Pre-record a backup run of the exact same demo data; if live fails, say so honestly and cut to the recording — judges respect transparency over pretending
Gemini misclassifies the hero DRIFT case unpredictably	Low-Med	High	Tune and lock the prompt against your exact demo data days before submission; don't change the demo script after that
Judges think "this is just a text diff"	Medium	High	Explicitly call out the ACCEPTABLE_ADLIB case in the demo — that's the proof it's semantic judgment, not string matching (a string diff would flag every ad-lib as broken)
IBM Bob usage looks like an afterthought	Medium	Med-High	Use it genuinely from hour 0, keep visible proof (commits, session logs), mention specifically in README what it helped build
Running out of time, pipeline half-broken	Medium	High	Follow Section 10 timeline strictly; Person D empowered to call "cut this feature" at hour 20 if anything is unstable
Real film script copyright issue	Low	High	Use only self-written sample script (Section 5) — never a real copyrighted screenplay

15. GLOSSARY (for teammates less familiar with the stack)

- **Kafka / Confluent topic:** think of it as a live, ordered mailbox — one system drops messages in (the "producer"), another system reads them as they arrive (the "consumer"). We use it because dialogue transcripts naturally arrive one take at a time, in order, while shooting is happening.
- **ADK (Agent Development Kit):** Google's framework for building multi-step AI agents that call tools and make decisions, instead of just answering one prompt.
- **Firestore:** a real-time database — when one part of our system writes to it, any screen watching that data updates instantly, which is what makes our dashboard feel "live."
- **Semantic diff:** comparing two pieces of text for *meaning* difference, not just character/word difference — this is why we need an LLM (Gemini) instead of a basic string-compare function.
- **State machine:** a fixed set of steps a process moves through in order (Section 2) — deterministic means every run follows the same steps, which is what makes a demo reliable and repeatable instead of random.

16. FINAL PRE-SUBMISSION CHECKLIST

- ☐ Public repo live, LICENSE present, README complete
- ☐ Hosted app reachable from a fresh incognito browser
- ☐ Confluent topic + producer/consumer confirmed working live, not just in a recorded video
- ☐ IBM Bob usage proof included
- ☐ Demo video $\leq 3:00$, rehearsed at least 3x, matches Section 9 exactly
- ☐ Backup demo recording ready in case live call fails on stage
- ☐ Devpost submission fully filled out (Section 13)
- ☐ Sample script contains no real copyrighted material
- ☐ All 4 teammates can explain the full workflow end-to-end, not just their own piece

Good luck. Start with Person A's pipeline — everything else is scaffolding around it.